

🔗 Perguntas Teóricas Possíveis — Recurso BD 2025/2026

Premissa: O professor **NÃO repete** as perguntas da Época Normal no Recurso.
Este ficheiro exclui todas as perguntas que saíram no Exame Teórico da Época Normal 2025/2026.

✗ Perguntas JÁ ELIMINADAS (Saíram na Época Normal 2025/2026)

#	Tema	Pergunta EN 25/26
1	Definição de Termos	Defina: BD, SGBD (componentes), Metadados
2	LDD vs LMD	Diferenças entre LDD e LMD + operações
3	Vistas vs Relações Base	O que é uma vista + diferenças com relação base
4	Funções Agregação + NULLs	Restrições das funções de agregação + efeito dos NULLs
5	Mecanismo de Resolução de Vistas	Como funciona o mecanismo de resolução de vistas
6	Técnicas de Descoberta de Factos	Propósito e descrição de cada técnica

⚠ **Nota:** A Pergunta 7 (Normalização de Fatura) e a Pergunta 8 (Modelação + SQL + Álgebra) saem **SEMPRE**, mas com enunciados diferentes (documento/fatura e cenário novos).

☑ PERGUNTAS COM MAIOR PROBABILIDADE DE SAIR NO RECURSO

💧💧💧 **PRIORIDADE MÁXIMA** (Frequência Altíssima + Não saíram na EN 25/26)

🔗 P1 — Integridade Referencial + ON DELETE / ON UPDATE

Frequência: 8+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 1

Pergunta esperada:

Explique o que entende por Integridade Referencial. Identifique e descreva quais as ações que se podem utilizar nas subcláusulas ON DELETE e ON UPDATE.

Resposta Rápida:

A **Integridade Referencial** é uma regra fundamental do modelo relacional que garante a consistência lógica entre tabelas relacionadas. Estabelece que se um registo numa tabela filha referencia um registo numa tabela pai (através de uma Chave Estrangeira - FK), o valor referenciado tem de existir como Chave Primária (PK) na tabela pai ou, em alternativa, ser nulo (NULL).

Para gerir a eliminação ou atualização de chaves primárias na tabela pai, o SQL disponibiliza as seguintes ações configuráveis nas cláusulas ON DELETE e ON UPDATE:

- **CASCADE:** As alterações propagam-se automaticamente. Apagar ou atualizar uma PK na tabela pai elimina ou atualiza correspondentemente todas as linhas filhas associadas.
- **SET NULL:** A chave estrangeira (FK) nas linhas filhas é alterada para NULL quando o registo pai é apagado/atualizado (requer que a coluna admita nulos).
- **SET DEFAULT:** A chave estrangeira (FK) nas linhas filhas é alterada para o valor padrão (Default) configurado para essa coluna.
- **RESTRICT:** Impede imediatamente a eliminação ou atualização do registo pai se existirem registos filhos associados.
- **NO ACTION:** Semelhante ao RESTRICT. Rejeita a operação no registo pai se houver registos filhos. A diferença técnica é que alguns SGBDs permitem adiar esta verificação para o final da transação (*deferred check*), enquanto o RESTRICT verifica e bloqueia de imediato.

🔗 P2 — Normalização: Objetivos e Impacto no Desempenho

Frequência: 8+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 2

Pergunta esperada:

No contexto do modelo relacional, quais os objetivos da normalização de dados? De que forma o processo de normalização poderá afetar, posteriormente, o desempenho da implementação?

Resposta Rápida:

A **Normalização** é uma técnica formal de desenho de bases de dados relacionais que visa:

1. **Minimizar a redundância** de dados para poupar espaço de armazenamento.
2. **Eliminar anomalias de atualização** (inserção, remoção e modificação).
3. **Garantir a consistência e integridade** lógica das relações.

O impacto da normalização no **desempenho** da aplicação é misto:

- **Operações de Leitura (Consultas/OLAP):** Prejudicado. A decomposição de uma tabela em várias tabelas menores obriga à realização de múltiplas operações de junção (**JOIN**), o que aumenta o custo computacional, a utilização de memória e o número de acessos ao disco (**I/O**).
- **Operações de Escrita (Inserções, Atualizações, Eliminações/OLTP):** Beneficiado. Como as tabelas são mais "estreitas" e a informação não está duplicada, as operações de escrita são muito mais rápidas, efetuadas num único local e envolvem menos bloqueios (**locks**) de tabelas e atualizações de índices.

Definições das Formas Normais:

- **Primeira Forma Normal (1FN):** Todos os atributos devem ser atômicos (valores indivisíveis, sem grupos repetidos ou arrays) e deve existir uma chave primária identificadora.
- **Segunda Forma Normal (2FN):** Está na 1FN e todos os atributos não chave dependem na totalidade da chave primária (elimina a dependência parcial, o que só se aplica a chaves primárias compostas).
- **Terceira Forma Normal (3FN):** Está na 2FN e nenhum atributo não chave depende de outro atributo não chave (elimina dependências transitivas).
- **Boyce-Codd Normal Form (BCNF):** Versão mais forte da 3FN. Uma tabela está na BCNF se, para qualquer dependência funcional $X \rightarrow Y$, X é uma superchave.

🔗 P3 — Anomalias de Atualização

Frequência: 6+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 3

Pergunta esperada:

Descreva os tipos de anomalias de atualização que podem ocorrer numa relação com dados redundantes. Dê exemplos.

Resposta Rápida:

Quando o esquema de uma base de dados não está devidamente normalizado, a redundância de dados dá origem a três tipos de anomalias de atualização:

- **Anomalia de Inserção:** Ocorre quando é impossível introduzir certas informações na base de dados sem que outra informação independente também seja inserida.
Exemplo: Numa tabela que mistura alunos e disciplinas, não é possível registar uma nova disciplina antes de ter pelo menos um aluno inscrito nela (caso a chave primária envolva o ID do aluno).
- **Anomalia de Remoção (Eliminação):** Ocorre quando a eliminação de um registo resulta na perda involuntária de outros dados distintos e importantes que deveriam ser mantidos.
Exemplo: Se eliminarmos o único aluno inscrito numa determinada disciplina, todos os dados sobre essa disciplina (como o nome do professor e créditos) são perdidos para sempre.
- **Anomalia de Modificação (Alteração):** Ocorre quando a atualização de um dado duplicado exige que se alterem múltiplos registos para evitar inconsistências. Se nem todas as cópias forem alteradas, os dados ficam num estado inconsistente.
Exemplo: Se o nome de um professor estiver guardado na linha de cada aluno, mudar o nome do professor exige alterar 100 registos. Se falhar um, o mesmo professor aparecerá com dois nomes diferentes.

🔗 P4 — Triggers de Bases de Dados

Frequência: 4+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 1

Pergunta esperada:

O que são triggers e para que servem? Quais as vantagens e desvantagens? Identifique os tipos quanto ao momento de execução.

Resposta Rápida:

Um **Trigger (Gatilho)** é um bloco de código procedural (armazenado na base de dados) que é executado (disparado) de forma automática pelo SGBD em resposta a eventos específicos como operações DML (**INSERT**, **UPDATE**, **DELETE**) numa tabela ou vista.

- **Propósitos:** Imposição de regras de negócio complexas que superam as restrições declarativas padrão (`CHECK`), automatização de auditorias (logs de alterações), manutenção de dados derivados/calculados e sincronização de tabelas relacionadas.
- **Vantagens:**
 - **Centralização:** A lógica de validação fica na BD, garantindo que qualquer aplicação (web, mobile, etc.) respeite as mesmas regras.
 - **Segurança e Auditoria:** Garante o registo de alterações de forma inviolável.
 - **Automatização:** Reduz o código repetitivo no lado da aplicação.
- **Desvantagens:**
 - **Opacidade (Dificuldade de Depuração):** A execução é implícita (invisível para a aplicação), tornando difícil rastrear erros e fluxos complexos.
 - **Desempenho:** Podem degradar a performance das escritas se executarem lógica pesada em cada linha alterada.
 - **Portabilidade:** A sintaxe dos triggers varia drasticamente entre SGBDs (MySQL, Oracle, SQL Server).
- **Tipos quanto ao Momento de Execução:**
 - **BEFORE:** Executa antes da operação DML ser gravada no disco (ideal para validação e modificação de dados de entrada).
 - **AFTER:** Executa após a operação DML ter sido validada e aplicada (ideal para auditoria e propagação de alterações).
 - **INSTEAD OF:** Executa em substituição da ação DML original (frequentemente usado em vistas não atualizáveis para direcionar os dados para as tabelas base).
- **Granularidade:** Podem ser de linha (`FOR EACH ROW` - corre para cada tuplo afetado) ou de instrução (`FOR EACH STATEMENT` - corre uma única vez por comando SQL).

PRIORIDADE ALTA (Frequência Moderada-Alta + Não saíram na EN 25/26)

P5 — Sistemas de BD vs Ficheiros + Vantagens/Desvantagens SGBD

Frequência: 7+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 3

Pergunta esperada:

Descreva as principais características de um Sistema BD e compare com os Sistemas de Ficheiros. Enuncie as vantagens e desvantagens de um SGBD.

Resposta Rápida:

Os **Sistemas de Ficheiros** tradicionais dispersam os dados por ficheiros independentes e o acesso é feito por programas específicos, gerando acoplamento entre a estrutura física e o código da aplicação. Os **Sistemas de BD** centralizam os dados com acesso intermediado pelo SGBD, mantendo um catálogo/dicionário que descreve os dados (metadados).

Diferenças Principais:

1. **Dependência de Dados:** Nos ficheiros, qualquer alteração na estrutura física exige reescrever o código da aplicação. Nos sistemas de BD, existe **Independência de Dados**.
2. **Redundância:** Nos ficheiros há redundância descontrolada. No SGBD, a redundância é controlada e centralizada.
3. **Concorrência e Transações:** SGBD possui mecanismos robustos para gerir acessos simultâneos sem corrupção (através do controlo de concorrência e propriedades ACID), enquanto os ficheiros oferecem suporte muito limitado.

Vantagens do SGBD:

- Controlo e redução da redundância de dados.
- Partilha concorrente e consistente de informação.
- Aplicação uniforme de restrições de integridade e segurança de acessos.
- Independência física e lógica dos dados.
- Mecanismos automáticos de cópia de segurança (backup) e recuperação de desastres.

Desvantagens do SGBD:

- **Complexidade:** Exige conhecimento especializado para instalação, configuração e administração (DBA).
- **Custo Inicial:** Elevados custos de licenças de software, hardware robusto e formação de pessoal.
- **Impacto da Falha:** A centralização toma o SGBD num ponto único de falha (*Single Point of Failure*) — se falhar, todas as aplicações param.

Quando utilizar Sistemas de Ficheiros:

Em aplicações muito simples, de utilizador único, com baixos volumes de dados, ou onde os recursos de hardware (memória e CPU) sejam extremamente limitados (ex: sistemas embebidos simples).

🔗 P6 — Arquitetura ANSI/SPARC (Nível Conceptual)

Frequência: 5+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 2

Pergunta esperada:

A arquitetura ANSI/SPARC identifica três níveis nos SGBD. Descreva pormenorizadamente o nível intermédio.

Resposta Rápida:

O nível intermédio da arquitetura ANSI/SPARC é o **Nível Conceptual**.

- **Descrição:** Representa a visão lógica global de toda a base de dados, unificando a perspetiva de todos os utilizadores. É a descrição estruturada dos dados guardados sem qualquer detalhe de armazenamento físico.
- **Conteúdo:** Define as entidades, atributos, relacionamentos, tipos de dados das colunas, restrições de integridade (como chaves primárias, estrangeiras e restrições CHECK) e regras de segurança de acesso.
- **Objetivo:** Funciona como uma camada de prevenção que liberta as aplicações do conhecimento do armazenamento físico. É neste nível que se define o modelo de dados lógico (ex: modelo relacional).

Esquemas da Arquitetura ANSI/SPARC (Contexto):

- **Esquema Externo (Nível Externo/Visão):** Define as vistas personalizadas para os utilizadores/aplicações (cada um vê apenas os dados relevantes para si).
- **Esquema Conceptual (Nível Conceptual):** Estrutura lógica unificada e global descrita acima.
- **Esquema Interno (Nível Interno/Físico):** Define como os dados são realmente gravados no disco, incluindo caminhos de ficheiros, tamanhos de blocos, índices e técnicas de compressão.

🔗 P7 — Independência de Dados

Frequência: 5+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 1

Pergunta esperada:

Descreva o conceito de independência de dados e a sua importância. Diferencie entre independência física e lógica.

Resposta Rápida:

A **Independência de Dados** é a capacidade de alterar o esquema de uma base de dados num determinado nível da arquitetura ANSI/SPARC sem a necessidade de reescrever os esquemas dos níveis superiores ou o código das aplicações. A sua importância reside na redução drástica dos custos de manutenção e na flexibilidade para evoluir o sistema.

Existem dois tipos:

- **Independência Física de Dados:** Refere-se à capacidade de modificar o esquema físico (interno) sem alterar o esquema conceptual ou lógico.
Exemplo: Mudar a BD para outro disco físico, alterar a estrutura de ficheiros, ou criar/remover índices para melhorar a performance de consultas, sem que as tabelas ou aplicações tenham de ser reescritas.
- **Independência Lógica de Dados:** Refere-se à capacidade de alterar o esquema conceptual sem que os esquemas externos (vistas de utilizadores) ou as aplicações precisem de ser modificados.
Exemplo: Adicionar uma nova tabela ou coluna, ou dividir uma tabela existente em duas (para normalização), mantendo o funcionamento das consultas antigas através da criação de vistas que simulam a estrutura anterior.

🔗 P8 — Data Warehouses: Benefícios e Problemas

Frequência: 4+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 1

Pergunta esperada:

Descreva os benefícios e problemas dos Data Warehouses. Distinga entre Data Warehouse e Data Mart.

Resposta Rápida:

Um **Data Warehouse (DW)** é um repositório de dados histórico, integrado, orientado por temas e não-volátil, especificamente concebido para apoiar os processos de tomada de decisão e análise estatística (OLAP).

- **Benefícios:**

- **Integração:** Consolida dados provenientes de fontes heterogêneas (diferentes BDs operacionais, folhas de cálculo) sob uma estrutura uniforme.
- **Perspetiva Histórica:** Mantém dados ao longo de anos para análise de tendências, ao contrário das BDs transacionais que focam no presente.
- **Performance:** Separa as consultas analíticas pesadas (OLAP) das bases de dados de produção (OLTP), evitando a lentidão destas últimas.

- **Problemas:**

- **Custo e Complexidade:** Projetos caros, demorados e com elevado risco de desvio de requisitos.
- **Processo ETL:** Extrair, Transformar e Carregar os dados das fontes para o DW é extremamente complexo devido a problemas de qualidade de dados.
- **Manutenção:** Requer atualizações contínuas sempre que as fontes de dados operacionais sofrem alterações de estrutura.

Diferença entre Data Warehouse e Data Mart:

- **Data Warehouse:** Repositório global que abrange os dados de toda a organização (visão corporativa completa).
- **Data Mart:** Um subconjunto do Data Warehouse focado exclusivamente num departamento ou área de negócio específica (exemplo: apenas vendas, ou apenas recursos humanos). É mais rápido e económico de implementar.

PRIORIDADE MODERADA (Menos frequentes mas presentes nos Modelos de Recurso)

P9 — LMD Procedimentais vs Não-Procedimentais

Frequência: 2+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 2

Pergunta esperada:

Explique as diferenças entre LMD procedimentais e não-procedimentais. Dê exemplos.

Resposta Rápida:

As Linguagens de Manipulação de Dados (LMD) dividem-se em dois tipos de acordo com a forma de obtenção dos dados:

- **LMD Procedimentais:** O utilizador especifica **o que** quer e detalha passo a passo **como** obter os dados através de algoritmos. Funcionam tipicamente num registo de cada vez (*one-record-at-a-time*), necessitando de ciclos de repetição e cursores.
Exemplos: Álgebra Relacional, linguagens procedimentais acopladas a SQL (PL/SQL na Oracle, T-SQL no SQL Server quando utilizam cursores e loops).
- **LMD Não-Procedimentais (Declarativas):** O utilizador apenas especifica **o que** quer obter, sem indicar os passos para a recolha física. O SGBD, através do seu otimizador de consultas, decide qual o melhor caminho de execução. Funcionam sobre conjuntos de registos de cada vez (*set-at-a-time*).
Exemplos: SQL (especificamente comandos `SELECT`), Cálculo Relacional.

P10 — Subquery vs Junção

Frequência: 3+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 1

Pergunta esperada:

Qual a diferença entre uma subquery e uma junção? Em que situações não é possível usar uma subquery?

Resposta Rápida:

- **Subquery (Subconsulta):** É uma consulta `SELECT` aninhada dentro de outra instrução SQL (como no `WHERE`, `HAVING`, `FROM` ou `SELECT` principal). É tipicamente utilizada para filtrar dados da consulta principal.
- **Junção (Join):** É uma operação que combina colunas de duas ou mais tabelas num único resultado consolidado, com base em colunas comuns (chaves primárias e estrangeiras).

Situações em que não é possível usar apenas uma subquery (necessitando de `JOIN`):

1. **Exibição de Atributos de Tabelas Distintas:** Quando o resultado final da consulta precisa de mostrar, no `SELECT` principal, colunas pertencentes a diferentes tabelas. Uma subquery colocada no `WHERE` apenas serve para filtrar a tabela principal, mas não consegue projetar as colunas da tabela interna na resposta final.
2. **Eficiência de Junções Múltiplas:** Embora teoricamente possível com subqueries correlacionadas pesadas, junções são a única via prática e otimizada quando se pretende relacionar muitas tabelas simultaneamente e expor múltiplos dados cruzados.

🔗 P11 — Arquitetura Cliente-Servidor (2 vs 3 Níveis)

Frequência: 3+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 2

Pergunta esperada:

Compare a arquitetura cliente-servidor de 2 e 3 níveis. Qual a mais adequada para a Web?

Resposta Rápida:

- **Arquitetura de 2 Níveis (2-tier):** O cliente (geralmente um programa instalado - *fat client*) comunica diretamente com o servidor de bases de dados. A interface gráfica e a lógica de negócio (regras de decisão) correm no computador do cliente, enquanto o servidor apenas processa consultas SQL e armazena os dados.
- **Arquitetura de 3 Níveis (3-tier):** Introduce uma camada intermédia (Servidor de Aplicação/Web Server). O cliente é leve (*thin client* - ex: um browser). A lógica de negócio está centralizada no Servidor de Aplicação, que faz a ponte comunicando com a base de dados (Servidor de BD).

Mais adequada para a Web: A arquitetura de 3 níveis.

- **Vantagens na Web:**
 1. **Segurança:** O utilizador final não tem acesso direto nem credenciais da BD (esta fica protegida atrás da firewall da aplicação).
 2. **Escalabilidade:** Permite pooling de conexões à BD (partilha de ligações em vez de abrir uma por utilizador) e balanceamento de carga no servidor de aplicação.
 3. **Manutenção:** Atualizações de lógica de negócio são feitas no servidor centralizado, sem necessidade de atualizar programas nos computadores dos clientes.

🔗 P12 — Atributos no Modelo Entidade-Relacionamento

Frequência: 3+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 2

Pergunta esperada:

Descreva os atributos num diagrama ER. Dê exemplos de simples, compostos, multi-valor e derivados.

Resposta Rápida:

Atributos são propriedades ou características que descrevem e qualificam uma Entidade ou um Relacionamento no modelo ER. Podem ser classificados como:

- **Simple (Atômicos):** Indivisíveis. Contêm apenas um valor unitário.
Exemplo: NIF ou Sexo. Representados por uma elipse simples.
- **Compostos:** Podem ser divididos em sub-atributos menores e independentes.
Exemplo: Morada (decomponível em Rua, Código Postal e Localidade). Representados por uma elipse ligada a outras elipses.
- **Multi-valorados (Multivalorados):** Podem conter um conjunto de valores para a mesma entidade.
Exemplo: Telefone (uma pessoa pode ter vários números) ou Email. Representados por uma elipse com contorno duplo.
- **Derivados:** O seu valor não é armazenado diretamente, mas sim calculado a partir de outros atributos ou dados do sistema.
Exemplo: Idade (calculada a partir da Data de Nascimento) ou TotalFatura. Representados por uma elipse tracejada.
- **Chave (Identificador):** Atributo cujos valores identificam de forma única cada ocorrência da entidade. Representado por texto sublinhado dentro da elipse.

🔗 P13 — Cursores SQL

Frequência: 2+ exames | Aparece nos Modelos de Recurso: ☒ Modelo 2

Pergunta esperada:

O que são cursores SQL? Qual o propósito? Descreva o ciclo de vida.

Resposta Rápida:

Um **Cursor** é uma estrutura de controlo disponibilizada pelas extensões procedimentais do SQL (como PL/SQL e T-SQL) que funciona como um apontador para um conjunto de linhas devolvido por uma consulta.

- **Propósito:** Permitir o processamento de registos linha a linha, ao contrário do comportamento padrão do SQL que é orientado a conjuntos. É necessário quando a lógica requer cálculos ou validações específicas para cada tuplo individualmente.

Ciclo de Vida de um Cursor:

1. **DECLARE:** Define o nome do cursor e associa-o à consulta `SELECT` cujos registos pretende ler.
2. **OPEN:** Executa a consulta associada, aloca memória e posiciona o cursor antes da primeira linha.
3. **FETCH:** Recupera os valores da linha corrente para variáveis e avança o cursor para a linha seguinte. É usado dentro de um ciclo até que todas as linhas sejam processadas.
4. **CLOSE:** Fecha o cursor, libertando o conjunto de resultados ativos e quaisquer bloqueios de concorrência, mas mantém a sua estrutura definida na memória.
5. **DEALLOCATE (ou liberação):** Remove permanentemente a definição do cursor da memória e liberta todos os recursos do sistema.

★ PRIORIDADE SECUNDÁRIA (Menos frequentes, mas presentes em Modelos de Recurso específicos)

🔗 P14 — Materialização de Vistas (Indexed/Materialized Views)

Aparece nos Modelos de Recurso: ☒ Modelo 1

Pergunta esperada:

Explique o conceito de materialização de vistas. Vantagens e desvantagens vs vistas tradicionais? Em que contextos é recomendável?

Resposta Rápida:

Ao contrário de uma vista tradicional (que é apenas uma consulta guardada em texto e reexecutada sempre que é chamada), uma **Vista Materializada** (ou Vista Indexada) tem o resultado da sua consulta fisicamente calculado e armazenado em disco, tal como se tratasse de uma tabela normal.

- **Vantagens:**
 - **Velocidade de Leitura:** Acelera drasticamente consultas complexas que envolvem agregações pesadas (`SUM`, `AVG`), ordenações ou junções de várias tabelas, pois o resultado já está pré-calculado.
- **Desvantagens:**
 - **Perda de Performance na Escrita:** Sempre que as tabelas base (que dão origem à vista) são modificadas (`INSERT`, `UPDATE`, `DELETE`), o SGBD precisa de recalcular e atualizar a vista materializada, gerando overhead.
 - **Armazenamento:** Consome espaço físico em disco.
- **Contextos Recomendáveis:**
 - Ambientes de **Data Warehouse e Business Intelligence (OLAP)** com grandes volumes de dados.
 - Tabelas de dados com elevada taxa de leitura e taxas de atualização/escrita muito baixas.

🔗 P15 — Operações de Junção (Theta, Equi, Natural, Outer, Semi)

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Descreva as diferenças entre Theta Join, Equijoin, Natural Join, Outer Join e Semijoin.

Resposta Rápida:

- **Theta Join (\$R \bowtie_{\{\theta\}} S\$):** Operação geral de junção que combina linhas de duas relações com base numa condição contendo qualquer operador de comparação (`=`, `>`, `<`, `\geq`, `\leq`, `\neq`).
- **Equijoin:** Caso particular do Theta Join onde a condição de junção utiliza exclusivamente o operador de igualdade (`=`). Mantém colunas com o mesmo nome duplicadas no resultado.
- **Natural Join (\$R \bowtie S\$):** Equijoin automático que compara colunas com o mesmo nome em ambas as tabelas. Ao contrário do Equijoin, remove colunas duplicadas do resultado final automaticamente.

- **Outer Join (Junção Externa - Esquerda, Direita ou Completa):** Devolve todos os registos correspondentes (Inner Join) e, adicionalmente, preserva os registos de uma ou de ambas as tabelas que não encontraram correspondência, preenchendo as colunas em falta com `NULL`.
- **Semijoin (\$R ltimes S\$):** Devolve apenas as linhas da primeira tabela (\$R\$) que encontram correspondência na segunda tabela (\$S\$), mas sem expor atributos da segunda tabela nem duplicar linhas da primeira. Equivale a filtrar com `EXISTS`.

🔗 P16 — Stored Procedures vs Funções (UDF)

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Diferença entre Stored Procedure e User-Defined Function? Três diferenças fundamentais.

Resposta Rápida:

Embora ambos sejam blocos de código guardados no SGBD, distinguem-se pelas seguintes diferenças fundamentais:

Característica	Funções (User-Defined Functions - UDF)	Stored Procedures (Procedimentos)
Retorno de Valores	Obrigatório: Devolve um único valor ou uma tabela (<code>RETURN</code>).	Opcional: Pode não devolver nada ou usar parâmetros de saída (<code>OUT</code>).
Invocação	Integradas em comandos SQL (ex: <code>SELECT col, func() FROM tab</code>).	Invocadas isoladamente com comandos próprios (ex: <code>EXEC</code> ou <code>CALL</code>).
Modificação de Dados	Apenas Leitura: Não podem alterar dados nem tabelas (DML bloqueado).	Escrita: Podem efetuar <code>INSERT</code> , <code>UPDATE</code> e <code>DELETE</code> .
Transações	Não é permitido usar <code>COMMIT</code> ou <code>ROLLBACK</code> .	Permitem o controlo e gestão total de transações.

🔗 P17 — Sublinguagens de Dados (DDL, DML, DCL, TCL)

Aparece nos Modelos de Recurso: ☒ Modelo 3

Pergunta esperada:

O que são sublinguagens de dados? Identifique e descreva DDL, DML, DCL, TCL com exemplos.

Resposta Rápida:

As sublinguagens de dados são divisões funcionais da linguagem SQL, agrupando comandos de acordo com o tipo de operação que realizam na base de dados:

- **DDL (Data Definition Language - Definição):** Comandos que definem, alteram ou eliminam as estruturas (esquemas) da base de dados.
Exemplos: `CREATE` (tabelas, vistas, índices), `ALTER`, `DROP`, `TRUNCATE`.
- **DML (Data Manipulation Language - Manipulação):** Comandos que permitem interagir com os dados armazenados nas tabelas.
Exemplos: `SELECT` (consulta), `INSERT` (inserção), `UPDATE` (alteração), `DELETE` (remoção).
- **DCL (Data Control Language - Controlo):** Comandos para gestão de acessos, segurança e privilégios dos utilizadores.
Exemplos: `GRANT` (atribuir privilégios), `REVOKE` (retirar privilégios).
- **TCL (Transaction Control Language - Transações):** Comandos para gerir os limites e estados das transações.
Exemplos: `COMMIT` (gravar alterações), `ROLLBACK` (cancelar alterações), `SAVEPOINT` (ponto intermédio).

🔗 P18 — 5 Operações Básicas da Álgebra Relacional

Aparece nos Modelos de Recurso: ☒ Modelo 3

Pergunta esperada:

Defina as cinco operações básicas da Álgebra Relacional. Demonstre como Junção e Interseção são derivadas.

Resposta Rápida:

As cinco operações básicas e independentes que definem a álgebra relacional são:

1. **Seleção (σ):** Operação unária que filtra as linhas (tuplos) que satisfazem uma determinada condição.
2. **Projeção (π):** Operação unária que seleciona colunas (atributos) específicas de uma relação, eliminando linhas duplicadas.
3. **Produto Cartesiano ($\times$):** Operação binária que combina todas as linhas da primeira relação com todas as linhas da segunda.

4. **União ($\cup$):** Operação binária que combina todas as linhas de duas tabelas (requer que as tabelas sejam compatíveis em número e tipo de colunas).
5. **Diferença ($-$):** Operação binária que devolve as linhas que estão na primeira relação mas não na segunda (requer compatibilidade).

Operações Derivadas (com demonstração matemática):

- **Junção (Join - $\bowtie$):** Combinação filtrada de duas tabelas. É derivada do Produto Cartesiano seguido de uma Seleção.
$$R \bowtie S \equiv \sigma_{condição}(R \times S)$$
- **Interseção ($\cap$):** Devolve as linhas comuns a duas tabelas. É derivada usando a operação de Diferença.
$$R \cap S \equiv R - (R - S)$$

P19 — Componentes do Ambiente de um SGBD

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Descreva os 5 componentes principais do ambiente de um SGBD.

Resposta Rápida:

O funcionamento global de um SGBD baseia-se na interação de cinco componentes principais:

1. **Hardware:** A infraestrutura física que suporta o sistema, incluindo o servidor (CPU, memória RAM), dispositivos de armazenamento físico (discos SSD/HDD) e equipamentos de rede.
2. **Software:** O conjunto de programas, composto pelo próprio software do SGBD (motor da base de dados), o Sistema Operativo onde este corre e as aplicações cliente que acedem aos dados.
3. **Dados:** O componente mais importante. Engloba tanto os dados operacionais dos utilizadores como os metadados (dicionário de dados/catálogo que descreve a estrutura física e lógica da BD).
4. **Procedimentos:** As regras administrativas, instruções e políticas de funcionamento da base de dados (ex: regras para fazer cópias de segurança/backup, formas de iniciar sessão e procedimentos de recuperação de falhas).
5. **Utilizadores:** As pessoas envolvidas no sistema: Administradores de BD (DBAs), Projetistas/Designers de bases de dados, Programadores de aplicações e os Utilizadores Finais (que consomem a informação).

P20 — Conceitos do Modelo Relacional

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Explique: Relação, Atributo, Domínio, Tuplo, Grau e Cardinalidade.

Resposta Rápida:

- **Relação:** Uma tabela bidimensional de dados que contém linhas e colunas. Matematicamente, representa um subconjunto do produto cartesiano dos domínios dos seus atributos.
- **Atributo:** Uma coluna nomeada numa relação. Representa uma propriedade ou característica dos dados.
- **Domínio:** O conjunto de todos os valores atómicos e válidos permitidos para um determinado atributo (ex: inteiros positivos, datas válidas, strings de 10 caracteres).
- **Tuplo:** Uma linha na relação. Representa um registo individual completo (uma ocorrência de dados).
- **Grau:** O número total de atributos (colunas) que constituem a estrutura de uma relação.
- **Cardinalidade:** O número total de tuplos (linhas) presentes numa relação num determinado momento (varia dinamicamente com inserções e eliminações).

P21 — Vistas Atualizáveis

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Quais as restrições para uma vista ser atualizável diretamente via DML?

Resposta Rápida:

Para que o SGBD consiga propagar sem ambiguidade operações DML (`INSERT`, `UPDATE`, `DELETE`) executadas sobre uma vista diretamente para as respetivas tabelas base, a vista tem de cumprir restrições restritas:

- **Tabela Base Única:** Deve mapear exatamente **uma única tabela base** no seu `FROM` (sem junções `JOIN`).
- **Sem Agregações:** A cláusula `SELECT` não pode conter funções de agregação (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`).
- **Sem Agrupamento ou Filtros de Grupo:** Não pode conter as cláusulas `GROUP BY` ou `HAVING`.
- **Sem Eliminação de Duplicados:** Não pode utilizar a palavra-chave `DISTINCT`.
- **Sem Operações de Conjuntos:** Não pode utilizar `UNION`, `INTERSECT` ou `EXCEPT`.
- **Atributos Obrigatórios:** Para inserções (`INSERT`), a vista deve incluir todas as colunas definidas como `NOT NULL` e sem valor por omissão (`DEFAULT`) na tabela base.
- **Sem Colunas Calculadas:** Não pode conter expressões ou atributos derivados na lista de seleção.

🔗 P22 — Especialização vs Generalização (Modelo ER)

Aparece nos Modelos de Recurso: ☒ Modelo 4

Pergunta esperada:

Diferenças entre especialização e generalização no diagrama ER. Dê exemplos.

Resposta Rápida:

Ambos os conceitos gerem a hierarquia de herança no modelo ER, distinguindo-se pelo sentido do processo de modelação:

- **Especialização (Abordagem Top-Down / De cima para baixo):** Processo que consiste em identificar sub-entidades (subclasses) específicas a partir de uma entidade global (superclasse), com base em características ou funções distintas.
Exemplo: A partir da entidade `Funcionário`, especializam-se as subclasses `Programador` e `Gestor` (apenas `Programador` tem o atributo "LinguagemFavorita").
- **Generalização (Abordagem Bottom-Up / De baixo para cima):** Processo inverso que consiste em identificar características comuns em várias entidades distintas e agrupá-las numa única entidade genérica (superclasse).
Exemplo: Ao analisar as entidades `Carro`, `Mota` e `Camião`, agrupam-se os atributos comuns (`Matrícula`, `Marca`) numa superclasse denominada `Veículo`.

Restrições associadas: A hierarquia pode ser definida quanto à **participação** (Total ou Parcial) e quanto à **disjunção** (Disjunta ou Sobreposta).

🔗 P23 — Transações e Propriedades ACID

Coberto nos resumos do professor

Pergunta esperada:

O que é uma transação? Descreva as propriedades ACID.

Resposta Rápida:

Uma **Transação** é uma unidade lógica de processamento que agrupa um conjunto de operações de base de dados (leituras/escritas) que devem ser executadas como um bloco único.

Para garantir a integridade dos dados face a falhas e acessos concorrentes, o SGBD deve assegurar as quatro propriedades **ACID**:

- **Atomicidade (Atomicity):** Princípio do "tudo ou nada". Todas as operações da transação são executadas com sucesso (`COMMIT`) ou, se ocorrer qualquer falha, nenhuma delas é aplicada, revertendo a BD ao estado inicial (`ROLLBACK`).
- **Consistência (Consistency):** A transação deve levar a base de dados de um estado consistente para outro estado consistente, respeitando todas as regras e restrições de integridade (chaves primárias, restrições referenciadas, etc.).
- **Isolamento (Isolation):** A execução de uma transação concorrente deve ocorrer de forma isolada das restantes. O resultado de transações simultâneas deve ser igual ao de uma execução sequencial das mesmas.
- **Durabilidade (Durability):** Uma vez concluída a transação com sucesso (`COMMIT`), as suas alterações tomam-se permanentes na base de dados e não podem ser perdidas, mesmo em caso de falha de energia ou colapso do sistema.

🔗 P24 — Abordagens para Múltiplas Vistas de Utilizadores

Frequência: 3+ exames

Pergunta esperada:

Enuncie as abordagens para desenho de BD com múltiplas vistas de utilizadores.

Resposta Rápida:

Ao projetar bases de dados corporativas de grande dimensão, diferentes grupos de utilizadores (departamentos) têm visões distintas dos requisitos de dados. As três abordagens para lidar com este desafio de desenho são:

1. **Abordagem Centralizada (Integração de Requisitos):** Os requisitos de todos os utilizadores são combinados e fundidos numa única lista de requisitos global. O esquema global da BD é desenhado diretamente a partir dessa lista única. Indicado para sistemas mais simples.
2. **Abordagem de Integração de Vistas (View Integration):** É desenhado um esquema de base de dados local independente para cada departamento/vista de utilizador. Posteriormente, estes esquemas locais são fundidos/integrados num esquema concetual global unificado, resolvendo conflitos de nomes e estruturas.
3. **Abordagem Mista:** Combina características das anteriores. Os requisitos fundamentais e comuns a todos os departamentos são desenhados de forma centralizada de imediato, enquanto as vistas altamente específicas são tratadas como esquemas locais integrados na fase final.

Resumo: Probabilidade por Pergunta para o Recurso

Prioridade	Pergunta	Tema
🔥🔥🔥	P1	Integridade Referencial + ON DELETE/UPDATE
🔥🔥🔥	P2	Normalização: Objetivos + Desempenho
🔥🔥🔥	P3	Anomalias de Atualização
🔥🔥🔥	P4	Triggers (Definição + Vantagens/Desvantagens)
🔥🔥	P5	Sistemas BD vs Ficheiros + SGBD
🔥🔥	P6	Arquitetura ANSI/SPARC (Nível Conceptual)
🔥🔥	P7	Independência de Dados
🔥🔥	P8	Data Warehouses
🔥	P9	LMD Procedimentais vs Não-Procedimentais
🔥	P10	Subquery vs Junção
🔥	P11	Arquitetura Cliente-Servidor (2 vs 3)
🔥	P12	Atributos no Modelo ER
🔥	P13	Cursors SQL
★	P14	Materialização de Vistas
★	P15	Operações de Junção (5 tipos)
★	P16	Stored Procedures vs Funções (UDF)
★	P17	Sublinguagens (DDL, DML, DCL, TCL)
★	P18	5 Operações Básicas de Álgebra Relacional
★	P19	Componentes do Ambiente SGBD
★	P20	Conceitos do Modelo Relacional
★	P21	Vistas Atualizáveis
★	P22	Especialização vs Generalização (ER)
★	P23	Transações e ACID
★	P24	Abordagens Múltiplas Vistas

💡 **Dica final:** O exercício de **normalização** (P7 no exame, vale 3 val.) e a **modelação com SQL + Álgebra Relacional** (P8, vale 5 val.) saem **SEMPRE** — mas com documentos e cenários diferentes. Pratica com os exames modelo de recurso!

Exames modelo de recurso disponíveis para praticar:

- [Modelo 1 \(/exames%20modelo/Exame_Modelo_Recurso_1_2025_2026.md\)](#) — TecnoShop + Companhia Aérea
- [Modelo 2 \(/exames%20modelo/Exame_Modelo_Recurso_2_2025_2026.md\)](#) — AutoFlex Rent-a-Car + Ginásio
- [Modelo 3 \(/exames%20modelo/Exame_Modelo_Recurso_3_2025_2026.md\)](#) — Grand Plaza Hotel + Reparação Eletrónica
- [Modelo 4 \(/exames%20modelo/Exame_Modelo_Recurso_4_2025_2026.md\)](#) — Clínica Geral do Norte + Stock Peças